barclaycard | payments

Martian Robots

This tech test is the practical stage of our process. It's a small, self-contained problem (below) that you complete in your own time. The problem itself is deliberately modest — what we're really assessing is *how you work*: how you structure code, make trade-offs, test your assumptions, and communicate your thinking.

Read the guidance here before you start, then work through the problem in the Description section.

How to approach it

- **Treat it like production code.** Clear structure, meaningful names, sensible error handling, and tests you'd actually want if this were shipping to customers. Include a short README covering how to build and run it, how to run the tests, and any decisions or assumptions you made.
- **But don't over-egg it.** We are *not* looking for a framework, a microservice, a database, or config for fifteen environments. Solve the problem well and stop. Knowing when to stop is part of the signal — the brief hints that new command types might be needed in future, so show you've kept that door open without building things nobody asked for.
- **Make it easy to run.** We should be able to clone your repo and get it running with minimal setup and clear instructions. If it doesn't run for us, it doesn't count for much.
- **Test the interesting cases.** The edge cases are where the problem lives — lost robots, the "scent" that stops later robots falling off the same edge, and instructions that are safely ignored as a result.
- **Use the language you'll do your best work in.** Our product stack is Rust and Kotlin on the back-end and TypeScript on the front-end, and using one of those is welcome — but a strong solution in the language you're most fluent in is absolutely fine.
- **Time-box it.** This shouldn't eat your weekend. If you'd have done something more given more time, just say so in the README — clear thinking about what you *didn't* do tells us as much as the code you wrote.

Using AI

- **You're encouraged to use AI tools** — just as you would on the job. We use them here every day, and pretending otherwise would be dishonest about how modern engineering works.
- **Your name stays on it.** You own every line you submit: you should understand it, be able to explain it, defend the trade-offs, and extend it live if we ask in the follow-up conversation. "The AI wrote that part" is not an answer we accept.
- **Use AI to raise your bar, not to replace your judgement.** It's there to help you go faster and produce your best work — not to submit code you couldn't have written or don't fully understand.

Submitting your work

- **Push it to GitHub** once you're done — a public repo, or a private one with access shared with us — and send us the link.
- **Include the README** described above: build/run instructions, how to run the tests, and a note on your key decisions and trade-offs.

barclaycard | payments

Description

The Problem

The surface of Mars can be modelled by a rectangular grid around which robots are able to move according to instructions provided from Earth. You are to write a program that determines each sequence of robot positions and reports the final position of the robot.

A robot position consists of a grid coordinate (a pair of integers: x-coordinate followed by y-coordinate) and an orientation (N, S, E, W for north, south, east, and west). A robot instruction is a string of the letters "L", "R", and "F" which represent, respectively, the instructions:

- Left: the robot turns left 90 degrees and remains on the current grid point.
- Right: the robot turns right 90 degrees and remains on the current grid point.
- Forward: the robot moves forward one grid point in the direction of the current orientation and maintains the same orientation.

The direction North corresponds to the direction from grid point (x, y) to grid point (x, y+1).

There is also a possibility that additional command types may be required in the future andprovision should be made for this.

Since the grid is rectangular and bounded (...yes Mars is a strange planet), a robot that moves "off" an edge of the grid is lost forever. However, lost robots leave a robot "scent" that prohibits future robots from dropping off the world at the same grid point. The scent is left at the last grid position the robot occupied before disappearing over the edge. An instruction to move "off" the world from a grid point from which a robot has been previously lost is simply ignored by the current robot.

The Input

The first line of input is the upper-right coordinates of the rectangular world, the lower-left coordinates are assumed to be 0, 0.

The remaining input consists of a sequence of robot positions and instructions (two lines per robot). A position consists of two integers specifying the initial coordinates of the robot and an orientation (N, S, E, W), all separated by whitespace on one line. A robot instruction is a string of the letters "L", "R", and "F" on one line.

Each robot is processed sequentially, i.e., finishes executing the robot instructions before the next robot begins execution.

The maximum value for any coordinate is 50.

All instruction strings will be less than 100 characters in length.

The Output

For each robot position/instruction in the input, the output should indicate the final grid position and orientation of the robot. If a robot falls off the edge of the grid the word "LOST" should be printed after the position and orientation.

barclaycard | payments

Sample Input

None

```
5 3
1 1 E
RFRFRFRF
3 2 N
FRRFLLFFRRFLL
0 3 W
LLFFFLFLFL
```

Sample Output

None

```
1 1 E
3 3 N LOST
2 3 S
```